

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Exp No: 1

Date: 10-07-2026

Name: Kushaal Shyam Potta

Reg No.: 3122235001071

Implementation of lexical analyser and symbol table

Aim:

To build a lexical analyzer (scanner) in C that reads a C++ source program, recognizes and classifies its tokens (keywords, identifiers, constants, operators, special characters, comments and function calls), and records the details of each identifier in a symbol table.

Tools Used:

- C language
- GCC

Code:

scanner.c

```

/* -----
Lexical Analyzer + Symbol Table Builder (for C++ programs)
Reads source.cpp, classifies tokens, builds symbol table.
----- */

#include <stdio.h>

#include <string.h>

#include <ctype.h>


#define MAX_SRC 6000

#define MAX_TOK 64

#define MAX_SYMS 100


static char src[MAX_SRC];

```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
static int srcLen = 0;
```

```
static int pos = 0;
```

```
typedef struct {
```

```
    char name[32];
```

```
    char type[12];
```

```
    int bytes;
```

```
    int address;
```

```
    char value[32];
```

```
} SymEntry;
```

```
static SymEntry table[MAX_SYMS];
```

```
static int symCount = 0;
```

```
static int addrCounter = 1000;
```

```
static const char *reserved[] = {
```

```
    "auto", "bool", "break", "case", "catch", "char", "class", "const", "constexpr",
```

```
    "continue", "default", "delete", "do", "double", "else", "enum", "explicit",
```

```
    "extern", "false", "float", "for", "friend", "goto", "if", "inline", "int", "long",
```

```
    "mutable", "namespace", "new", "nullptr", "operator", "private", "protected",
```

```
    "public", "register", "return", "short", "signed", "sizeof", "static", "struct",
```

```
    "switch", "template", "this", "throw", "true", "try", "typedef", "typename",
```

```
    "union", "unsigned", "using", "virtual", "void", "volatile", "while"
```

```
};
```

```
static const int reservedCount = sizeof(reserved) / sizeof(reserved[0]);
```

```
static int sizeOfType(const char *t) {
```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (!strcmp(t, "char")) return 1;
    if (!strcmp(t, "bool")) return 1;
    if (!strcmp(t, "float")) return 4;
    if (!strcmp(t, "double")) return 8;
    return 2;
}

static int isReserved(const char *word) {
    for (int i = 0; i < reservedCount; i++)
        if (!strcmp(word, reserved[i])) return 1;
    return 0;
}

static void recordSymbol(const char *name, const char *type) {
    if (symCount >= MAX_SYMS) return;
    strcpy(table[symCount].name, name);
    strcpy(table[symCount].type, type);
    table[symCount].bytes = sizeofType(type);
    table[symCount].address = addrCounter;
    addrCounter += table[symCount].bytes;
    strcpy(table[symCount].value, "-");
    symCount++;
}

static int peek(int offset) {
    int i = pos + offset;
    return (i < srcLen) ? src[i] : '\0';
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

}

```
static void skipBlanksFrom(int *idx) {
    while (*idx < srcLen && isspace((unsigned char)src[*idx])) (*idx)++;
}
```

```
static void handlePreprocessor(void) {
    printf("#");
    pos++;
    while (pos < srcLen && src[pos] != '\n') putchar(src[pos++]);
    printf(" - preprocessor directive\n");
}
```

```
static int handleComment(void) {
    if (peek(0) == '/' && peek(1) == '/') {
        while (pos < srcLen && src[pos] != '\n') pos++;
        return 1;
    }
    if (peek(0) == '/' && peek(1) == '*') {
        pos += 2;
        while (pos < srcLen && !(src[pos] == '*' && peek(1) == '/')) pos++;
        if (pos < srcLen) pos += 2;
        else printf("Error: unterminated comment\n");
        return 1;
    }
    return 0;
}
```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
static char activeType[12] = "";
```

```
static void handleWordToken(void) {
```

```
    char word[MAX_TOK];
```

```
    int i = 0;
```

```
    while (isalnum((unsigned char)src[pos]) || src[pos] == '_')
```

```
        word[i++] = src[pos++];
```

```
    word[i] = '\0';
```

```
    int look = pos;
```

```
    skipBlanksFrom(&look);
```

```
    if (isReserved(word)) {
```

```
        printf("%s - keyword\n", word);
```

```
        if (!strcmp(word,"int") || !strcmp(word,"char") || !strcmp(word,"float") ||
```

```
            !strcmp(word,"double") || !strcmp(word,"long") || !strcmp(word,"short") ||
```

```
            !strcmp(word,"bool"))
```

```
            strcpy(activeType, word);
```

```
        else
```

```
            activeType[0] = '\0';
```

```
        return;
```

```
    }
```

```
    if (src[look] == '(') {
```

```
        printf("%s", word);
```

```
        pos = look;
```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

while (pos < srcLen && src[pos] != ')') putchar(src[pos++]);
if (pos < srcLen) { putchar(src[pos]); pos++; }
printf(" - function call\n");
activeType[0] = '\0';
return;
}

```

```

printf("%s - identifier\n", word);
if (activeType[0] != '\0') {
    recordSymbol(word, activeType);
    int r = pos;
    skipBlanksFrom(&r);
    if (src[r] == '=') {
        printf("= - assignment operator\n");
        r++;
        skipBlanksFrom(&r);
        char val[MAX_TOK]; int j = 0;
        while (r < srcLen && src[r] != ';' && src[r] != ';') val[j++] = src[r++];
        val[j] = '\0';
        printf("%s - constant\n", val);
        strcpy(table[symCount - 1].value, val);
        pos = r;
    }
}
}
}

```

```

static void handleNumberToken(void) {

```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```
char num[MAX_TOK];
```

```
int i = 0;
```

```
if (src[pos] == '0' && (peek(1) == 'x' || peek(1) == 'X')) {
```

```
    num[i++] = src[pos++];
```

```
    num[i++] = src[pos++];
```

```
    while (isxdigit((unsigned char)src[pos])) num[i++] = src[pos++];
```

```
    num[i] = '\0';
```

```
    printf("%s - hexadecimal constant\n", num);
```

```
    return;
```

```
}
```

```
while (isdigit((unsigned char)src[pos])) num[i++] = src[pos++];
```

```
if (src[pos] == '.') {
```

```
    num[i++] = src[pos++];
```

```
    while (isdigit((unsigned char)src[pos])) num[i++] = src[pos++];
```

```
    num[i] = '\0';
```

```
    printf("%s - double constant\n", num);
```

```
    return;
```

```
}
```

```
num[i] = '\0';
```

```
printf("%s - integer constant\n", num);
```

```
}
```

```
static void handleStringToken(void) {
```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

char str[100];

int i = 0;

str[i++] = src[pos++];

while (pos < srcLen && src[pos] != "" && src[pos] != '\n' && i < 98) {

    if (src[pos] == '\\' && pos + 1 < srcLen) str[i++] = src[pos++]; /* escape */

    str[i++] = src[pos++];

}

if (src[pos] == "") str[i++] = src[pos++];

str[i] = '\0';

printf("%s - string constant\n", str);

}

```

```

static void handleCharToken(void) {

    char ch[8];

    int i = 0;

    ch[i++] = src[pos++];

    while (pos < srcLen && src[pos] != '\"' && src[pos] != '\n' && i < 6) {

        if (src[pos] == '\\' && pos + 1 < srcLen) ch[i++] = src[pos++];

        ch[i++] = src[pos++];

    }

    if (src[pos] == '\"') ch[i++] = src[pos++];

    ch[i] = '\0';

    printf("%s - character constant\n", ch);

}

```

```

static int handleTwoCharOperator(void) {

    char c0 = src[pos], c1 = peek(1);

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

if ((c0=='+'&&c1=='=') || (c0=='-'&&c1=='=') || (c0=='*&&c1=='=') ||
    (c0=='/'&&c1=='=') || (c0=='%'&&c1=='=')) {
    printf("%c%c - arithmetic assignment operator\n", c0, c1); pos += 2; return 1;
}
if ((c0=='+'&&c1=='') || (c0=='-'&&c1=='')) {
    printf("%c%c - unary operator\n", c0, c1); pos += 2; return 1;
}
if (c0=='&&'&&c1=='&') { printf("&& - logical operator\n"); pos+=2; return 1; }
if (c0=='|'&&c1=='|') { printf("|| - logical operator\n"); pos+=2; return 1; }
if (c0=='='&&c1=='=') { printf("== - relational operator\n"); pos+=2; return 1; }
if (c0=='!'&&c1=='=') { printf("!= - relational operator\n"); pos+=2; return 1; }
if (c0=='<'&&c1=='=') { printf("<= - relational operator\n"); pos+=2; return 1; }
if (c0=='>'&&c1=='=') { printf(">= - relational operator\n"); pos+=2; return 1; }
if (c0=='<'&&c1=='<') { printf("<< - bitwise / stream insertion operator\n"); pos+=2; return 1; }
if (c0=='>'&&c1=='>') { printf(">> - bitwise / stream extraction operator\n"); pos+=2; return 1; }
if (c0==':'&&c1==':') { printf(":: - scope resolution operator\n"); pos+=2; return 1; }
if (c0=='-'&&c1=='>') { printf("-> - member access operator\n"); pos+=2; return 1; }
return 0;
}

```

```

static void handleSingleCharOperator(void) {
    char c = src[pos];
    if (c=='+' || c=='-' || c=='*' || c=='/' || c=='%') { printf("%c - arithmetic operator\n", c); pos++; return; }
    if (c=='<' || c=='>') { printf("%c - relational operator\n", c); pos++; return; }
    if (c=='=') { printf("=" - assignment operator\n"); pos++; return; }
    if (c=='!') { printf("! - logical operator\n"); pos++; return; }
}

```

Course Code	UCS2702		Course Name	Compiler Design	
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    if (c=='?' || c==':')          { printf("%c - conditional operator\n", c); pos++; return; }
    if (c=='&' || c=='|' || c=='^') { printf("%c - bitwise operator\n", c); pos++; return; }
}

```

```

static int handleSpecialChar(void) {
    if (strchr(";,.[>(){}\"", src[pos]) != NULL) {
        printf("%c - special character\n", src[pos]);
        if (src[pos] == ';') activeType[0] = '\0';
        pos++;
        return 1;
    }
    return 0;
}

```

```

static void printSymbolTable(void) {
    printf("\nContent of Symbol Table\n");
    printf("%-15s %-8s %-12s %-10s %-10s\n", "Identifier", "Type", "No of bytes", "Address", "Value");
    for (int k = 0; k < symCount; k++)
        printf("%-15s %-8s %-12d %-10d %-10s\n",
            table[k].name, table[k].type, table[k].bytes, table[k].address, table[k].value);
}

```

```

static int loadSource(const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (!fp) return 0;
    srcLen = (int)fread(src, 1, MAX_SRC - 1, fp);
    src[srcLen] = '\0';
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

fclose(fp);

return 1;
}

int main(void) {
    if (!loadSource("source.cpp")) {
        printf("cannot open source.cpp\n");
        return 1;
    }

    while (pos < srcLen) {
        if (isspace((unsigned char)src[pos])) { pos++; continue; }
        if (src[pos] == '#') { handlePreprocessor(); continue; }
        if (handleComment()) continue;

        if (isalpha((unsigned char)src[pos]) || src[pos] == '_') { handleWordToken(); continue; }
        if (isdigit((unsigned char)src[pos])) { handleNumberToken(); continue; }
        if (src[pos] == '"') { handleStringToken(); continue; }
        if (src[pos] == '\\') { handleCharToken(); continue; }
        if (handleTwoCharOperator()) continue;

        if (strchr("+-*/%<>=!&|^?:", src[pos])) { handleSingleCharOperator(); continue; }
        if (handleSpecialChar()) continue;

        pos++;
    }
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

    printSymbolTable();

    return 0;
}

source.cpp:

#include <iostream>

using namespace std;

// single line comment

/* multi
   line comment */

int main()
{
    int x = 0x1F, count = 5;

    float pi = 3.14;

    char ch = 'A';

    bool done = false;

    x += 2;

    count++;

    if (x <= 10 && count != 0) {
        cout << "ok\n" << ch << endl;
    }

    while (x > 0) {
        x--;
    }

    return 0;
}

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

Output:

```

● kushaal@Kushaals-MacBook-Air Assignment1 % gcc scanner.c -o scanner
● kushaal@Kushaals-MacBook-Air Assignment1 % ./scanner
#include <iostream> - preprocessor directive
using - keyword
namespace - keyword
std - identifier
; - special character
int - keyword
main() - function call
{ - special character
int - keyword
x - identifier
= - assignment operator
0x1F - constant
, - special character
count - identifier
= - assignment operator
5 - constant
; - special character
float - keyword
pi - identifier
= - assignment operator
3.14 - constant
; - special character
char - keyword
ch - identifier
= - assignment operator
'A' - constant
; - special character
bool - keyword
done - identifier
= - assignment operator
false - constant
; - special character
x - identifier
+= - arithmetic assignment operator
2 - integer constant
; - special character
count - identifier
++ - unary operator
; - special character
if - keyword
( - special character
x - identifier
<= - relational operator
10 - integer constant
&& - logical operator
count - identifier
!= - relational operator
0 - integer constant

```

Course Code	UCS2702	Course Name	Compiler Design		
Regulation	2021	Batch	2023-27	Academic Year	2026-27 (ODD)

```

) - special character
{ - special character
cout - identifier
<< - bitwise / stream insertion operator
"ok\n" - string constant
<< - bitwise / stream insertion operator
ch - identifier
<< - bitwise / stream insertion operator
endl - identifier
; - special character
} - special character
while - keyword
( - special character
x - identifier
> - relational operator
0 - integer constant
) - special character
{ - special character
x - identifier
-- - unary operator
; - special character
} - special character
return - keyword
0 - integer constant
; - special character
} - special character

```

Content of Symbol Table

Identifier	Type	No of bytes	Address	Value
x	int	2	1000	0x1F
count	int	2	1002	5
pi	float	4	1004	3.14
ch	char	1	1008	'A'
done	bool	1	1009	false

Learning Outcomes:

- Understood that lexical analysis is the first phase of a compiler and what it is responsible for.
- Learned how the different kinds of tokens in a programming language are recognized and classified.
- Learned how to build and maintain a symbol table that stores the attributes of identifiers.
- Got practical experience in detecting lexical errors such as unterminated comments and strings.